INSTITUTO MILITAR DE ENGENHARIA (IME)

Programação Aplicada à Engenharia Elétrica (SE/3 - Eletrônica)

Guia Didático Passo a Passo: Entregável 1

Subsistema IIO no Linux Embarcado, Modelagem POO em C++ e Doxygen

Prazo Oficial (Classroom):	**Segunda-feira, 26/10/2026 às 23:59** (Valor: **2,0 pts** / 800 pts)
Kit de Desenvolvimento:	STMicroelectronics **STM32MP157D-DK1** (Linux OpenSTLinux)
Sensor do Grupo (Nº 3):	Módulo de Obstáculo Infravermelho Reflexivo Digital (**HW-201** / LM393)
Objetivo da Entrega 1:	Classe em C++ para leitura do sensor documentada com Doxygen (via link do commit final no GitHub)

1 Cronograma Confirmado e Avaliação da Entrega 1

Pelo novo edital atualizado em *15 de outubro de 2026* e pelas tarefas publicadas no Google Classroom, as datas oficiais de todas as etapas estão confirmadas (**atenção especial:** as Entregas 4 e 5 vencem juntas no dia **16/11/2026**):

Etapa	Prazo (23:59)	Valor	Descrição Oficial no Classroom / Edital
Entrega 0	**19/10/2026** (Seg)	Obrig.	Foto da conexão física do sensor com o kit **desligado**.
Entrega 1	**26/10/2026** (Seg)	2,0 pts	Código-fonte em C++ da classe de leitura documentada com Doxygen.
Entrega 2	**02/11/2026** (Seg)	2,0 pts	Programa funcional no kit + instruções de compilação/execução no `README.md`.
Entrega 3	**09/11/2026** (Seg)	2,0 pts	Envio da leitura via UDP para o servidor + documentação do protocolo.
Entrega 4	**16/11/2026** (Seg)	2,0 pts	Interface gráfica no PC (tempo real, histórico 30–60s e log/CSV).
Entrega 5	**16/11/2026** (Seg)	2,0 pts	**Mesmo dia da Entrega 4!** Vídeo de demonstração, apresentação e documentação completa no `README.md`.



Figura 1: Tarefa oficial da Entrega 1 publicada no Google Classroom (26/10 às 23:59).

As 3 Regras de Ouro da Avaliação (Evite Penalizações)

1. **Proibição Estrita de Variáveis Globais e Funções Soltas (Regra nº 2):**
   Por que o professor exige isso? Porque na Entrega 5 (Simulação Integrada), o código dos

11 grupos da turma poderá ser integrado em um único sistema. Se dois grupos criarem uma variável global `int leitura;` ou uma função solta `lerSensor()`, haverá colisão de nomes (*linker error*). Quando tudo está **encapsulado dentro da sua classe C++** (ex.: `class IrObstacleSensor`), cada sensor vive isolado dentro do seu próprio objeto!

2. **Divisão Profissional de Arquivos (`.hpp` e `.cpp`):**
   Em C++, não colocamos tudo dentro de um único arquivo. Criamos um **arquivo de cabeçalho** (`include/IrObstacleSensor.hpp`) para declarar *o que a classe tem e faz*, e um **arquivo de implementação** (`src/IrObstacleSensor.cpp`) para programar *como cada método funciona*.

3. **Documentação com Doxygen e Link do Commit Final:**
   A classe deve conter comentários especiais `/** ... */` acima da classe, dos atributos e dos métodos. A entrega no Google Classroom deve ser o **link direto do commit final** no GitHub do grupo.

2 A Teoria: Como o Linux da Placa Lê um Pino Físico?

Se você já programou em Arduino, deve lembrar da função `analogRead(A0)`. Mas na sua placa **STM32MP157D-DK1**, o código não roda "direto no ferro" (*bare-metal*): ele roda dentro de um **Sistema Operacional Linux**.

No Linux, "Tudo é um Arquivo" (*Everything is a File*)

No Linux, um programa comum de usuário é proibido de acessar registradores de memória física diretamente por questões de segurança. Em vez disso, o Kernel do Linux possui um driver chamado **IIO (*Industrial I/O*)** que conversa com o conversor Analógico-Digital (ADC) do chip STM32MP157D e cria "arquivos virtuais" na pasta:

`/sys/bus/iio/devices/iio:device0/`

Dentro dessa pasta existem arquivos chamados `in_voltage0_raw`, `in_voltage1_raw`, ..., `in_voltageX_raw`.
A mágica é a seguinte: Toda vez que o seu programa em C++ abre esse arquivo como se fosse um `.txt` comum (usando `std::ifstream`) e lê o número escrito lá dentro, o Kernel do Linux faz uma leitura elétrica instantânea no pino do ADC e devolve um número inteiro (por exemplo, `0` quando a tensão é 0 V, ou um valor alto como `4095` / `65535` quando a tensão é 3,3 V)!

3 Passo 1 (No Laboratório): Descobrir o Canal `X` do ADC

Depois que o professor aprovar a sua foto da Entrega 0 e autorizar ligar a placa, você precisa descobrir qual número `X` no nome do arquivo `in_voltageX_raw` corresponde ao **Pino 5 do conector de 6 furos** onde você espetou o fio de sinal do seu sensor.

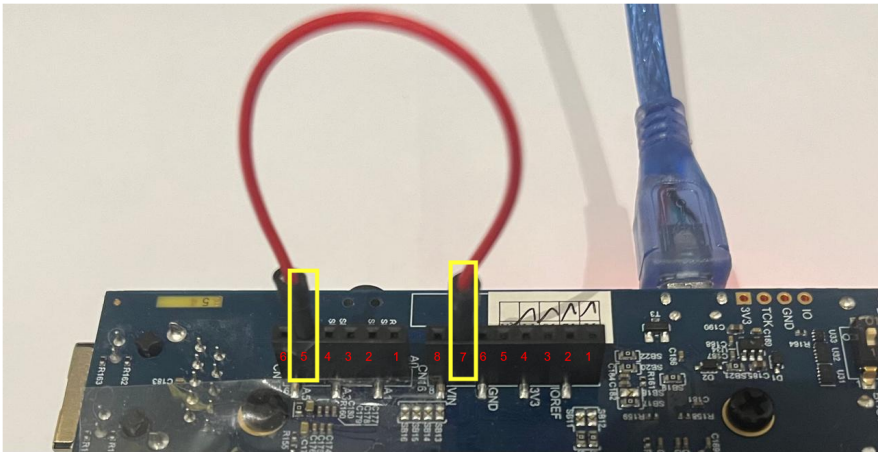


Figura 2: Figura 3 do roteiro oficial: Jumper ligando diretamente o Pino 5 do conector ADC (6 furos à esquerda) ao Pino 6 de `GND` do conector de alimentação (8 furos à direita) para forçar leitura zero.

Roteiro Prático no Laboratório (10 minutos)

1. **Aterrar o pino do ADC (Conforme Figura 3 acima):**
   Com a placa desligada, retire temporariamente o fio de sinal do sensor do **Pino 5 do conector de 6 furos** e use um cabo jumper Macho-Macho para ligar esse **Pino 5 (ADC)** diretamente ao **Pino 6 ou 7** (`GND`) do bloco de 8 furos ao lado. Isso garante fisicamente que a tensão naquele pino seja exatamente **0 V**.

2. **Ligar a Placa e Abrir o Terminal Serial:**
   - Conecte o cabo **USB-C** na porta `PWR_IN` (ao lado do conector Ethernet) para alimentar a placa.
   - Conecte o cabo **Micro-USB** na porta serial da placa (`ST-LINK`) e no computador do laboratório.
   - Abra o programa **TeraTerm** (no Windows) ou **minicom** (no Linux), selecione a conexão **Serial** e configure: **Baudrate = 115200**, **8N1** (8 bits, sem paridade, 1 stop bit).
   - Aperte `Enter`. Quando pedir login, digite usuário `root` e senha `root`.

3. **Inspecionar os Canais IIO no Terminal da Placa:**
   Digite os comandos abaixo no terminal da STM32MP157D-DK1:

```
# Entra na pasta do conversor A/D (subsistema IIO)
cd /sys/bus/iio/devices/iio:device0/

# Lista todos os arquivos disponiveis na pasta
ls

# Mostra o nome de cada canal junto com o valor lido nele
head in_voltage*_raw
```

4. **Anote o resultado:** O arquivo `in_voltageX_raw` que exibir o valor `0` (ou muito próximo de zero, ex.: `1` ou `2`) enquanto os outros exibem números altos é o seu canal!

5. **Teste rápido com o Sensor HW-201 conectado:** Desligue a placa, reconecte os 3 fios do sensor HW-201 (igual à Entrega 0), ligue a placa novamente e rode `cat in_voltageX_raw` duas vezes:
   - **Sem nada na frente do sensor:** Anote o valor alto lido (ex.: `4095` ou `65535`).
   - **Com a mão ou tampa a 5 cm do sensor:** Veja o LED do sensor acender e anote o valor baixo lido (próximo de `0`). A média entre esses dois números será o seu **limiar** (`threshold`) perfeito!

4 Passo 2 (Em Casa): Como Simular a Placa no seu PC

Você não precisa estar no laboratório para programar e testar sua classe C++. Como o programa apenas lê um arquivo de texto contendo um número inteiro, você pode criar um "sensor falso" (*mock*) no seu computador!

Criando seu Simulador Local em 30 Segundos

No terminal do seu Linux (ou WSL / Git Bash em casa), crie um arquivo de texto para fingir que é o canal IIO da placa:

```
# Cria uma pasta temporaria para simular o diretorio da placa
mkdir -p /tmp/mock_iio

# Escreve um valor alto (simulando 3,3V -> Sem obstaculo / Tampa aberta
    )
echo "4095" > /tmp/mock_iio/in_voltage0_raw

# Quando quiser simular um obstaculo/tampa fechada (0V), basta rodar:
echo "120" > /tmp/mock_iio/in_voltage0_raw
```

Dica de Arquitetura: Se você programar o construtor da sua classe C++ para receber o **caminho do arquivo como parâmetro** (deixando o caminho real `"/sys/bus/iio/devices/iio:device0/in_voltageX_raw"` como valor padrão), o seu código funcionará em casa e na placa sem precisar mudar uma única linha!

5 Passo 3 (Em Casa): Construindo sua Classe C++ do Zero

Para você aprender de verdade a matéria de Programação Orientada a Objetos (POO), vamos construir a sua classe etapa por etapa. Você mesmo vai criar dois arquivos:

1. `include/IrObstacleSensor.hpp` (O Cabeçalho / Contrato da Classe)
2. `src/IrObstacleSensor.cpp` (A Implementação / Lógica dos Métodos)

5.1 Projetando o Cabeçalho (`include/IrObstacleSensor.hpp`)

Abra o arquivo `include/IrObstacleSensor.hpp` em branco e siga estes 4 blocos conceituais:

Conceito 1: Include Guards (Proteção contra Inclusão Dupla)

Todo arquivo `.hpp` deve começar com `#ifndef NOME_HPP`, `#define NOME_HPP` e terminar com `#endif`. Isso impede que o compilador dê erro de "classe redefinida" caso dois arquivos `.cpp` façam `#include "IrObstacleSensor.hpp"` no futuro.

O que você deve declarar dentro da sua `class IrObstacleSensor`:

- **Na seção `private`: (Encapsulamento — ninguém de fora mexe diretamente):**
  1. Uma variável `std::string` (ex.: `devicePath`) para guardar o caminho do arquivo IIO.
  2. Uma variável `int` (ex.: `lastRawValue`) para guardar o último número lido do ADC.
  3. Uma variável `int` (ex.: `threshold`) para guardar o limiar que separa nível BAIXO (0 V) de nível ALTO (3,3 V).
- **Na seção `public`: (A interface que o resto do programa pode chamar):**
  1. **O Construtor:** Tem o mesmo nome da classe e não tem tipo de retorno. Use argumentos padrão (*default arguments*) para facilitar o uso:

```
explicit IrObstacleSensor(
    const std::string& path = "/sys/bus/iio/devices/iio:device0/
       in_voltage0_raw",
    int alertThreshold = 2000
);
```

  2. **Método de Leitura Bruta (`int readRaw();`):** Abre o arquivo, atualiza `lastRawValue` e retorna o inteiro lido.
  3. **Método de Interpretação Física (`bool isObstacleDetected() const;`):** Lembra que o sensor HW-201 é *Active-Low*? Quando há obstáculo/reflexão, a tensão cai para 0 V (`lastRawValue < threshold`).
  4. **Getters e Setters:** Ex.: `int getLastRawValue() const;` e `void setThreshold(int newThreshold);`. *Pergunta de prova:* Por que colocamos a palavra `const` no final de `getLastRawValue() const`? Porque isso promete ao compilador que esse método apenas lê dados e **não modifica** nenhum atributo da classe!

5.2 Programando a Lógica (`src/IrObstacleSensor.cpp`)

No arquivo `src/IrObstacleSensor.cpp`, você começa com `#include "IrObstacleSensor.hpp"`, `#include <fstream>` (para ler arquivos) e `#include <stdexcept>` (para lançar erros caso o arquivo não exista).

Conceito 2: Como implementar os métodos usando o operador de escopo `::`

Fora do `.hpp`, para avisar ao compilador que uma função pertence à sua classe, você escreve `NomeDaClasse::nomeDoMetodo(...)`. Veja como funciona a leitura de arquivo em C++ com `std::ifstream`:

```
int IrObstacleSensor::readRaw() {
    // 1. Cria um objeto ifstream tentando abrir o arquivo no caminho
       devicePath
    std::ifstream file(devicePath);

    // 2. Verifica se o arquivo abriu com sucesso
    if (!file.is_open()) {
        throw std::runtime_error("Erro ao abrir o canal IIO: " +
           devicePath);
    }

    // 3. Usa o operador de extracao (>>) para ler o inteiro do arquivo
        para o atributo
    file >> lastRawValue;

    // 4. Retorna o valor lido (o arquivo se fecha sozinho ao sair da
       funcao!)
    return lastRawValue;
}
```

Por que não precisamos dar `file.close()`? Porque em C++ moderno o destrutor de `std::ifstream` fecha o arquivo automaticamente assim que a função termina (princípio chamado **RAII**).

6 Passo 4 (Em Casa): Documentando com Doxygen

O edital exige que sua classe esteja documentada com **Doxygen**. O Doxygen é um programa que lê comentários especiais no formato `/** ... */` dentro do seu código e gera um site HTML completo de documentação.

Como Escrever Comentários Doxygen no seu `.hpp`

Coloque blocos `/** ... */` imediatamente acima da classe e de cada método no seu `IrObstacleSensor.hpp`, usando as tags principais:

- `@file`: Nome do arquivo e resumo do propósito dele (no topo do arquivo).
- `@class`: Nome da classe e explicação geral.
- `@brief`: Uma frase curta explicando o que o método ou atributo faz.
- `@param nome`: Explica para que serve cada parâmetro que o método recebe.
- `@return`: Explica o que o método devolve quando termina.

Exemplo de como documentar um método:

```
/**
 * @brief Avalia se o feixe infravermelho detectou um obstaculo ou
    reflexao.
 * @details No modulo HW-201 (LM393), a deteccao de reflexao leva a
    saida para 0V.
 * @return true se a leitura estiver abaixo do limiar (obstaculo
    presente), false caso contrario.
 */
bool isObstacleDetected() const;
```

Como testar a geração do Doxygen no terminal:

```
# 1. Gera um arquivo de configuracao padrao chamado Doxyfile
doxygen -g Doxyfile

# 2. (Dica) Abra o Doxyfile num editor de texto e mude a linha INPUT
   para:
#    INPUT = include src
#    E mude EXTRACT_PRIVATE = YES para documentar tambem seus atributos
    privados!

# 3. Rode o Doxygen para gerar a pasta html/
doxygen Doxyfile
```

Depois, abra o arquivo `html/index.html` no seu navegador e veja se todos os seus métodos aparecem documentados!

7 Passo 5: Compilando, Testando e Entregando (Até 26/10)

Para testar se a sua classe não tem nenhum erro de compilação antes de subir para o GitHub, crie um arquivo simples `src/main.cpp` que instancia um objeto `IrObstacleSensor` apontando para o seu arquivo simulado e imprime o resultado no terminal:

```
# Compila o seu codigo com todos os avisos ligados (-Wall -Wextra)
g++ -Wall -Wextra -std=c++17 -Iinclude src/IrObstacleSensor.cpp src/main.cpp
    -o teste_sensor

# Executa o seu programa
./teste_sensor
```

7.1 Checklist Final para o Commit da Entrega 1

☐ Verifique se **não existe nenhuma variável global** nem função solta fora da classe nos seus arquivos `.hpp` e `.cpp`.

☐ Atualize o `README.md` explicando a estrutura da sua classe `IrObstacleSensor`, seus atributos privados, seus métodos públicos e como compilar/gerar a documentação Doxygen.

☐ Faça o commit e suba para o GitHub:

```
git add include/ src/ Doxyfile README.md
git commit -m "Entrega 1: Implementacao da classe IrObstacleSensor com
   Doxygen"
git push origin main
```

☐ **Como pegar o Link do Commit Final exigido no Google Classroom:**
Entre na página do seu repositório no GitHub, clique em **"Commits"** (no canto superior direito da lista de arquivos), clique sobre o título do último commit que você acabou de enviar e **copie a URL completa do navegador** (formato `https://github.com/usuario/repo/commit/a1b2c3d4...`). Cole esse link na tarefa **[Projeto] Entrega 1** do Google Classroom!

7.2 Links Oficiais do Edital (Já Adiante para a Entrega 2 — 02/11)

Extraídos diretamente do PDF atualizado do professor (`Projeto_de_Programacao_Aplicada_2026 (1).pdf`):

- **Download da Toolchain ARM** (`arm-buildroot-linux-gnueabihf_sdk-DK2.tar.gz`):
  https://drive.google.com/drive/folders/1wr7YHFreOkxdnQLaE4kMThYUD1u1lBhI?usp=sharing
- **Tutorial Oficial de Criação da Máquina Virtual Ubuntu no VirtualBox:**
  https://ubuntu.com/tutorials/how-to-run-ubuntu-desktop-on-a-virtual-machine-using-virtualbox